

QMS Future Cloning and Deployment Guide

MacBook staging workflow for creating additional QMS Raspberry Pi units

Reference/Master Pi: 192.168.0.149

Master username: sonvisage

Master OS: Debian 12 (Bookworm), aarch64

QMS repository: /home/sonvisage/QMS_Pi_code

Mac staging folder: /Users/wealth/QMS_Pi_Deployment

MASTER QMS PI	MACBOOK	NEW QMS PI
192.168.0.149	QMS_Pi_Deployment	Fresh Raspberry Pi OS
Copy QMS + services	Keep deployment backup	Boot with unique hostname
	Transfer over SSH/SCP	Install QMS + services
		Connect to GitHub updates

1. Purpose of This Clone Method

This procedure creates a new QMS deployment unit without making an exact disk-for-disk clone. The MacBook stores a reusable deployment package from the working master Pi. Each new Raspberry Pi receives a fresh operating system, its own hostname and SSH identity, then the QMS software and services.

2. Create the Deployment Folder on the MacBook

```
mkdir -p ~/QMS_Pi_Deployment
cd ~/QMS_Pi_Deployment
pwd
```

Expected Mac path:

```
/Users/wealth/QMS_Pi_Deployment
```

3. Copy the QMS Project from the Master Pi

```
scp -r sonvisage@192.168.0.149:/home/sonvisage/QMS_Pi_code ~/QMS_Pi_Deployment/
```

Enter the password for the master Pi when requested. This command copies the working project, including its local Git metadata and runtime files. The copy is useful as a backup, but the new deployment Pi should not inherit the master's SSH private key.

4. Copy the systemd Service Files

```
scp sonvisage@192.168.0.149:/etc/systemd/system/mqtt_queue.service ~/QMS_Pi_Deployment/
scp sonvisage@192.168.0.149:/etc/systemd/system/qms-update.service ~/QMS_Pi_Deployment/
scp sonvisage@192.168.0.149:/etc/systemd/system/qms-update.timer ~/QMS_Pi_Deployment/
```

Copy the automatic updater script:

```
scp sonvisage@192.168.0.149:/usr/local/bin/qms-update.sh ~/QMS_Pi_Deployment/
```

Verify the deployment package size:

```
du -sh ~/QMS_Pi_Deployment
```

In the current setup the package was approximately 624 KB.

5. Files That Must Not Be Reused as Device Identity

Do not copy the master's private SSH key to a new Raspberry Pi. Each Pi should generate its own key. The new deployment unit should also have its own hostname and network identity.

```
DO NOT COPY:  
/home/sonvisage/.ssh/id_ed25519
```

Generate a new SSH key on each deployment Pi instead.

The live queues.json file represents runtime queue state. It is already ignored by the Git deployment workflow. For a clean deployment, start the new Pi with its own queue state rather than treating the master's live queue as permanent configuration.

6. Identify the New SD Card on the MacBook

```
diskutil list
```

Identify the external physical SD card by its size. In the example used during setup, the new card appeared as:

```
/dev/disk4 (external, physical)  
62.9 GB  
Windows_FAT_32 NO NAME
```

Important: /dev/disk0 and the synthesized internal APFS disks belong to the Mac. Never erase a disk until the external SD card has been positively identified.

7. Write Raspberry Pi OS

Use Raspberry Pi Imager on the MacBook. Select the actual Raspberry Pi model and an appropriate 64-bit Raspberry Pi OS. To minimize differences from the working master, use a Bookworm-based image when appropriate for the hardware and available in Raspberry Pi Imager.

Recommended OS customization for the first deployment unit:

```
Hostname: qms-pi-02  
Username: sonvisage  
Password: choose and record a device password  
Wi-Fi: configure the required network  
Wi-Fi country: NG  
Timezone: Africa/Lagos  
SSH: enabled  
Initial SSH authentication: password
```

Write the image to the confirmed external SD card. When imaging is complete, eject it before removal:

```
diskutil eject /dev/disk4
```

8. First Boot of the New QMS Pi

Insert the newly imaged SD card into the new Raspberry Pi, power it on, and allow it time to join the network.

Try resolving the configured hostname from the Mac:

```
ping qms-pi-02.local
```

Stop ping with Ctrl+C, then try SSH:

```
ssh sonvisage@qms-pi-02.local
```

After login, verify the new Pi:

```
hostname
hostname -I
whoami
uname -m
cat /etc/os-release | head
```

The hostname should be qms-pi-02 and the IP address should be different from the master Pi's 192.168.0.149.

9. Transfer the Mac Deployment Package to the New Pi

Once the new Pi's actual IP address is known, replace NEW_PI_IP in the example below. NEW_PI_IP is a placeholder, not a literal hostname.

```
scp -r ~/QMS_Pi_Deployment sonvisage@NEW_PI_IP:/home/sonvisage/
```

Example only, if the new Pi receives 192.168.0.155:

```
scp -r ~/QMS_Pi_Deployment sonvisage@192.168.0.155:/home/sonvisage/
```

10. Recreate the Python Environment

Do not rely on copying the master's Python virtual environment. Recreate mqtt-env on the new Pi so it matches the new operating system and architecture. Before deployment, obtain the working package list from the master:

```
/home/sonvisage/mqtt-env/bin/pip freeze
```

Save the required dependency list and install those packages into a new /home/sonvisage/mqtt-env on the deployment Pi.

11. Install the QMS Services on the New Pi

After transferring the deployment package and QMS code, install the service files in /etc/systemd/system and the updater in /usr/local/bin. Preserve root ownership for system files and user ownership for the QMS repository.

```
sudo cp ~/QMS_Pi_Deployment/mqtt_queue.service /etc/systemd/system/
sudo cp ~/QMS_Pi_Deployment/qms-update.service /etc/systemd/system/
sudo cp ~/QMS_Pi_Deployment/qms-update.timer /etc/systemd/system/
sudo cp ~/QMS_Pi_Deployment/qms-update.sh /usr/local/bin/qms-update.sh
sudo chmod 755 /usr/local/bin/qms-update.sh
sudo systemctl daemon-reload
```

12. GitHub Access for Deployment Pis

The development/master Pi can have read/write GitHub access. Field deployment Pis should normally have read-only repository access so they can download releases but cannot accidentally push changes.

```
Development devices:
READ = yes
WRITE = yes

Deployment Pis:
READ = yes
WRITE = no
```

Generate a separate SSH key on each deployment Pi. Do not reuse the master's private key.

13. Enable and Test the QMS

```
sudo systemctl enable mqtt_queue.service
sudo systemctl restart mqtt_queue.service
systemctl status mqtt_queue.service --no-pager

sudo systemctl enable --now qms-update.timer
systemctl status qms-update.timer --no-pager
```

Force an immediate update check:

```
sudo /usr/local/bin/qms-update.sh
```

Check logs if necessary:

```
journalctl -u mqtt_queue.service -n 50 --no-pager
journalctl -u qms-update.service -n 50 --no-pager
```

14. Normal Future Software Deployment

After the new Pi is deployed, normal updates no longer require repeating the cloning process. Edit and test the QMS on an authorized development device, then push the approved change to GitHub.

```
git status
git add mqtt_queue_manager.py
git commit -m "Describe the QMS update"
git push origin main
```

Each configured deployment Pi checks GitHub approximately every five minutes. When a new main-branch commit is detected, it fast-forwards to the new version and restarts `mqtt_queue.service`.

15. Quick Checklist for Every New QMS Pi

Check	Requirement
SD card	Fresh Raspberry Pi OS installed
Hostname	Unique, e.g. qms-pi-02, qms-pi-03
User	sonvisage
SSH	Enabled and tested
QMS code	Installed under <code>/home/sonvisage/QMS_Pi_code</code>
Python	mqtt-env recreated with required dependencies
Service	mqtt_queue.service enabled and running
Updater	qms-update.timer enabled and waiting
GitHub	Deployment Pi has its own read-only identity

Runtime data	queues.json is not overwritten by Git
Verification	MQTT connects and expected topics are subscribed

Result: Each QMS Pi remains an independent device while receiving centrally controlled software updates from GitHub.